

راهنمای کوتاه اجرای پروژه روی Kaggle

پروژه درس پردازش زبان طبیعی پیشرفته — پرسش و پاسخ چندمرحله‌ای با RAG و GraphRAG

استفاده از Kaggle اجباری نیست؛ این راهنما فقط برای کسانی است که GPU شخصی ندارند و می‌خواهند مدل زبانی باز را روی GPU رایگان Kaggle اجرا کنند. منوها و امکانات Kaggle گاهی تغییر می‌کنند؛ اگر چیزی با این راهنما نمی‌خواند، مستندات خود Kaggle مرجع است.

۱ ساختن نوت‌بوک و فعال کردن GPU و اینترنت

۱. در kaggle.com وارد شوید و از `Create > New Notebook` یک نوت‌بوک بسازید.
۲. برای استفاده از GPU و اینترنت، حساب شما باید با شماره تلفن **تأیید** شده باشد (Settings حساب کاربری).
۳. در پنل کناری نوت‌بوک (Session options / Settings): Accelerator را روی یکی از گزینه‌های GPU (مثلاً GPU T4 x2) بگذارید و Internet را روشن کنید (برای نصب کتابخانه‌ها و دانلود مدل از Hugging Face لازم است).
۴. با `!nvidia-smi` مطمئن شوید GPU در دسترس است و حافظه آن را ببینید.

۲ افزودن بسته داده درس

تیم درس بسته `course pack v1` را به صورت **یک دیتاست مشترک** در Kaggle منتشر می‌کند (نشانی آن در کانال درس اعلام می‌شود). از `Add Input` (یا `+ Add Data`) آن را جست‌وجو و به نوت‌بوک اضافه کنید. فایل‌ها زیر `/kaggle/input/` و فقط خواندنی قرار می‌گیرند:

```
!ls /kaggle/input/ # find the folder name of the course dataset
PACK = "/kaggle/input/<course-dataset>/course_pack_v1" # adjust to what ls shows
!ls {PACK} # corpus.jsonl, questions_*.jsonl, ...
```

۳ گرفتن کد از مخزن خصوصی GitHub

۱. در GitHub یک `fine-grained personal access token` بسازید که فقط به **همان یک مخزن** و فقط با دسترسی `Contents: Read-only` (یا `Read and write` اگر می‌خواهید از `push` Kaggle کنید) محدود باشد و تاریخ انقضا داشته باشد.
۲. در نوت‌بوک از `Add-ons > Secrets` یک راز با نام مثلاً `GH_TOKEN` بسازید و برای این نوت‌بوک فعالش کنید.
۳. مخزن را طوری `clone` کنید که توکن هیچ‌جا چاپ نشود:

```
import subprocess
from kaggle_secrets import UserSecretsClient
token = UserSecretsClient().get_secret("GH_TOKEN")
repo = "github.com/<your-user>/<your-private-repo>.git"
r = subprocess.run(["git", "clone", f"https://{token}@{repo}", "/kaggle/working/project"],
                    capture_output=True, text=True)
print("clone ok" if r.returncode == 0 else "clone failed") # never print r.stderr / the URL
subprocess.run(["git", "-C", "/kaggle/working/project", "remote", "set-url", "origin",
                f"https://{token}@{repo}"]) # remove the token from .git/config
del token
```

🔑 کلیدها و توکن‌ها را هرگز چاپ یا commit نکنید

توکن را در سلول‌ها ننویسید، چاپ نکنید و در هیچ فایل (از جمله خروجی‌های نوت‌بوک و `run_config.yaml`) ذخیره نکنید. نوت‌بوک‌های خود را خصوصی نگه دارید. اگر توکنی افشا شد، فوراً آن را در GitHub باطل کنید. همین قاعده برای توکن دسترسی Hugging Face (جهت استفاده از مدل‌های نیازمند مجوز دسترسی یا همان gated models) نیز برقرار است: آن را هم در بخش Secrets قرار دهید.

۴ اجرای اسکریپت‌ها

کد شما در `/kaggle/working/project` است و می‌توانید آن را مانند ترمینال با `!` اجرا کنید:

```
%cd /kaggle/working/project
!pip install -q -r requirements.txt
!python score.py runs/dev/dense/predictions.jsonl --pack {PACK} --split dev
!python src/my_pipeline.py --pack {PACK} --split hidden --out runs/hidden/dense/predictions.jsonl
```

(نام `src/my_pipeline.py` فقط مثال است؛ ساختار کد به انتخاب شماست.) تعبیه‌های پیکره را یک بار محاسبه و در `/kaggle/working` ذخیره کنید (مثلاً فایل `.npy`) تا در هر اجرا دوباره حساب نشوند.

۵ اجرای مدل زبانی باز روی GPU

حافظه: هر GPU مدل T4 حدود ۱۶ گیگابایت حافظه دارد. مدل‌های ۸ تا ۱۴ میلیارد پارامتری با دقت ۱۶ بیتی در آن جا نمی‌شوند (یا جایی برای زمینه نمی‌ماند)؛ از نسخه‌های **کوانتیزه‌شده ۴ بیتی** (مثلاً AWQ یا GPTQ برای vLLM، یا bitsandbytes برای transformers) استفاده کنید. T4 از bfloat16 پشتیبانی نمی‌کند؛ نوع داده را float16 بگذارید. نسخه‌های جدید vLLM ممکن است برخی GPUهای قدیمی‌تر (مثل P100) را پشتیبانی نکنند؛ پیش از نصب، سازگاری نسخه را بررسی کنید.

گزینه ۱: vLLM (سریع، مناسب اجرای دسته‌ای ۳۰۰ پرسش):

```
!pip install -q vllm # pin the version you tested in requirements.txt
from vllm import LLM, SamplingParams
llm = LLM(model="<org>/<open-model-up-to-14B>-AWQ", quantization="awq", dtype="half",
          max_model_len=8192, gpu_memory_utilization=0.90,
          tensor_parallel_size=1) # 2 to split the model over both T4s
params = SamplingParams(temperature=0, max_tokens=128)
outputs = llm.chat([{"role": "user", "content": p} for p in prompts], params)
answers = [o.outputs[0].text for o in outputs]
```

گزینه ۲: transformers (ساده‌تر و کندتر):

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
name = "<org>/<open-model-up-to-14B>"
bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_compute_dtype=torch.float16)
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForCausalLM.from_pretrained(name, quantization_config=bnb, device_map="auto")
```

همان مدلی که در فاز ۱ اعلام کرده‌اید (حداکثر حدود ۱۴ میلیارد پارامتر، وزن باز) را تا پایان پروژه نگه دارید و محل اجرا (مثلاً "Kaggle T4, vLLM, 4-bit AWQ") را در `run_config.yaml` بنویسید. اگر حافظه کم آمد، `max_model_len` یا بودجه زمینه را کم کنید، یا مدل کوچک‌تری انتخاب کنید.

۶ ذخیره و دریافت خروجی‌ها

- فقط پوشه `/kaggle/working` پس از پایان نشست قابل نگهداری است؛ هر چیز دیگری (از جمله مدل‌های دانلودشده در حافظه موقت) با بسته شدن نشست پاک می‌شود. فضای آن هم محدود است؛ وزن مدل‌ها را آنجا ذخیره نکنید.
- **Save Version:** گزینه `Save & Run All (Commit)` نوت‌بوک را از ابتدا در پس‌زمینه اجرا می‌کند و محتوای `/kaggle/working` را به‌عنوان Output آن نسخه نگه می‌دارد (برای اجراهای طولانی مناسب است، چون به باز بودن مرورگر نیاز ندارد). `Quick Save` فقط خود نوت‌بوک را ذخیره می‌کند.
- خروجی‌ها (مثلاً `predictions.jsonl`) را از پنل Output نسخه ذخیره‌شده یا از پنل فایل‌های نشست جاری دانلود کنید. برای چند فایل، ابتدا یک فایل فشرده بسازید: `!cd /kaggle/working && zip -r runs.zip project/runs`.
- کد را در GitHub نگه دارید، نه در Kaggle؛ شناسه `commit` گزارش باید به همان کدی اشاره کند که اجراها را تولید کرده است.

۷ نگهداری داده‌های MLflow (اختیاری)

```
!pip install -q mlflow==<same-version-as-on-your-laptop>
%cd /kaggle/working
import mlflow
mlflow.set_tracking_uri("sqlite:///kaggle/working/mlflow.db") # 4 slashes = absolute path
# artifacts of new experiments go to ./mlruns, i.e. /kaggle/working/mlruns
```

پس از پایان کار، `mlflow.db` (و پوشه `mlruns` در صورت ثبت فایل) را دانلود کنید و روی لپ‌تاپ با فرمان `mlflow ui` دیداست خصوصی (یا نسخه جدید آن) ذخیره کنید، آن را به نوت‌بوک اضافه کنید و پیش از شروع، فایل را از `/kaggle/input/...` به `/kaggle/working` کپی کنید (چون `input` فقط خواندنی است). `W&B` نیازی به این کارها ندارد: کافی است کلید آن را در `Secrets` بگذارید.

۸ سهیمه و محدودیت‌های GPU

🔧 سهیمه‌ها تغییر می‌کنند

Kaggle برای استفاده از GPU یک **سهیمه هفتگی** و برای هر نشست یک **سقف زمانی** دارد و نشست‌های بیکار پس از مدتی بسته می‌شوند. مقدار این محدودیت‌ها در طول زمان تغییر می‌کند؛ مقدار فعلی را در صفحه **سهیمه حساب کاربری خود** (و مستندات Kaggle) ببینید و به عدد‌های شنیده‌شده تکیه نکنید.

- پیش از اجرای کامل، خط لوله را روی ۱۰ تا ۲۰ پرسش آزمایش کنید (بخش زیادی از کار، مثل نوشتن معیارها و BM25، اصلاً به GPU نیاز ندارد؛ این‌ها را با `Accelerator: None` یا روی لپ‌تاپ انجام دهید).
- نتایج میانی (تعبیه‌ها، پاسخ‌های مدل برای هر پرسش) را مرتب در `/kaggle/working` ذخیره کنید تا قطع شدن نشست کار را از بین نبرد.
- وقتی کارتان تمام شد، نشست را **متوقف** کنید (`Stop Session`) تا سهیمه هدر نرود.
- برای اجراهای طولانی (مثلاً هر چهار سامانه روی `dev` و `hidden`) از `Save & Run All` استفاده کنید.